

LAPORAN PRAKTIKUM DAN TUGAS

STRUKTUR DATA

“STACK”

DISUSUN OLEH:

Aliyatar Rafi Ahmad

2511533031

DOSEN PENGAMPU:

Dr. Wahyudi, S.T, M.T

ASISTEN PRAKTIKUM:

Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga laporan praktikum Struktur Data Pekan 3 ini dapat diselesaikan dengan baik.

Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas praktikum Struktur Data. Pada praktikum pekan ketiga ini, saya mempelajari konsep **Stack** serta implementasinya dalam bahasa pemrograman Java.

Saya menyadari bahwa laporan ini masih memiliki kekurangan, oleh karena itu saya mengharapkan kritik dan saran yang membangun.

Saya juga mengucapkan terima kasih kepada asisten praktikum yang telah membimbing selama praktikum berlangsung.

Padang, 18 Apr. 26

Aliyatar Rafi Ahmad

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan.....	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 Pengertian Stack.....	2
2.2 Karakteristik Stack	2
2.3 Operasi pada Stack.....	2
2.4 Implementasi dalam Java	2
2.4.1 StackArray_2511533031.....	3
2.4.2 StackArrayDriver_2511533031	4
2.4.3 SiswaStack_2511533031	5
2.4.4 StackPostFix_2511533031	6
2.4.5 NilaiMaksimum_2511533031.....	7
BAB III PENUTUP	8
3.1 Kesimpulan	8
DAFTAR PUSTAKA	9
LAPORAN TUGAS.....	10

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan bagian penting dalam pemrograman untuk mengelola data secara efisien. Salah satu struktur data yang sering digunakan adalah **Stack**, yaitu struktur data yang menggunakan prinsip LIFO (Last In First Out), dimana elemen terakhir yang masuk akan menjadi elemen pertama yang keluar.

Stack banyak digunakan dalam berbagai aplikasi seperti manajemen memori, evaluasi ekspresi, dan proses undo-redo. Oleh karena itu, pemahaman tentang Stack sangat penting bagi mahasiswa dalam mempelajari struktur data.

Pada praktikum pekan ketiga ini, mahasiswa mempelajari konsep dasar Stack serta implementasinya dalam bahasa pemrograman Java.

1.2 Tujuan

1. Memahami konsep dasar Stack
2. Mampu mengimplementasikan Stack dalam Java
3. Melatih kemampuan dalam mengelola data menggunakan struktur LIFO

1.3 Manfaat

1. Menambah pemahaman tentang struktur data Stack
2. Melatih kemampuan pemrograman dalam Java
3. Memahami konsep LIFO dalam pengolahan data

BAB II

PEMBAHASAN

2.1 Pengertian Stack

Stack adalah struktur data linear yang menyimpan elemen dengan urutan tertentu, dimana operasi dilakukan berdasarkan prinsip **LIFO (Last In First Out)**. Artinya, elemen yang terakhir dimasukkan ke dalam stack akan menjadi elemen pertama yang dikeluarkan.

Struktur data ini sering digunakan dalam berbagai aplikasi seperti pengolahan ekspresi, rekursi, serta fitur undo dan redo pada aplikasi. Konsep stack juga dapat dianalogikan seperti tumpukan buku, dimana buku yang terakhir diletakkan akan diambil terlebih dahulu.

2.2 Karakteristik Stack

Stack memiliki beberapa karakteristik utama, yaitu:

1. Menggunakan prinsip LIFO (Last In First Out)
2. Penambahan dan penghapusan elemen hanya dilakukan pada satu sisi (top)
3. Bersifat sederhana dan efisien untuk kebutuhan tertentu

2.3 Operasi pada Stack

- **push:** Digunakan untuk menambahkan elemen ke bagian atas stack
- **pop:** Digunakan untuk menghapus elemen teratas dari stack
- **peek:** Digunakan untuk melihat elemen teratas tanpa menghapusnya
- **isEmpty:** digunakan untuk mengecek apakah stack kosong
- **size:** Digunakan untuk mengetahui jumlah elemen dalam stack

2.4 Implementasi dalam Java

Pada praktikum pekan ke-3 telah dibuat beberapa kode program yang berkaitan dengan implementasi struktur data **Stack** dalam bahasa pemrograman Java. Program-program tersebut bertujuan untuk memahami cara kerja stack serta penggunaan operasi-operasi dasarnya seperti push, pop, dan peek.

Berikut kode program yang telah dibuat:

2.4.1 StackArray_2511533031

```

1 package pekan3_2511533031;
2
3 public class stackArray_2511533031 {
4     static final int MAX_3031 = 1000;
5     int top_3031;
6     int a[] = new int [MAX_3031];
7     boolean isEmpty_2511533031()
8     {
9         return (top_3031 < 0);
10    }
11    stackArray_2511533031()
12    {
13        top_3031 = -1;
14    }
15    boolean push_3031(int x)
16    {
17        if (top_3031 >= (MAX_3031 - 1)) {
18            System.out.println(x + " dimasukkan dalam stack");
19            return false;
20        }
21        else {
22            a[++top_3031] = x;
23            System.out.println(x + " dimasukkan dalam stack");
24            return true;
25        }
26    }
27
28    int pop_3031()
29    {
30        if (top_3031 < 0) {
31            System.out.println("Stack Underflow");
32            return 0;
33        }
34        else {
35            int x = a[top_3031--];
36            return x;
37        }
38    }
39    int peak_3031()
40    {
41        if (top_3031 < 0) {
42            System.out.println("Stack Underflow");
43            return 0;
44        }
45        else {
46            int x = a[top_3031];
47            return x;
48        }
49    }
50    void print_3031() {
51        for (int i = top_3031; i > -1; i--) {
52            System.out.print(" " + a[i]);
53        }
54    }
55 }
56 }
57

```

Gambar 2.1

Program `stackArray_2511533031` merupakan implementasi struktur data **Stack** menggunakan array dalam Java. Stack memiliki kapasitas maksimum yang ditentukan oleh `MAX_3031`, dan posisi elemen teratas ditunjukkan oleh variabel `top_3031` yang diinisialisasi dengan `-1` saat stack kosong. Method `isEmpty` digunakan untuk mengecek apakah stack kosong.

Program ini menyediakan operasi utama stack yaitu `push` untuk menambah elemen, `pop` untuk menghapus elemen teratas, dan `peak` untuk

melihat elemen teratas. Selain itu, terdapat method print untuk menampilkan isi stack. Program juga menangani kondisi overflow dan underflow agar penggunaan stack tetap aman.

2.4.2 StackArrayDriver_2511533031

```

1 package pekan3_2511533031;
2 import java.util.*;
3 public class stackArrayDriver_2511533031 {
4
5     public static void main(String[] args) {
6         stackArray_2511533031 s= new stackArray_2511533031();
7         s.push_3031(10);
8         s.push_3031(20);
9         s.push_3031(30);
10        System.out.println(s.pop_3031() + " dikeluarkan dari stack");
11        System.out.println("Elemen teratas adalah :" + s.peak_3031());
12        System.out.print("Element pada stack :");
13        s.print_3031();
14    }
15 }
16
17 }

```

Gambar 2.2

Program stackArrayDriver_2511533031 merupakan program utama yang digunakan untuk menguji implementasi stack berbasis array. Pada awal program dibuat sebuah objek stack, kemudian dilakukan penambahan beberapa elemen yaitu 10, 20, dan 30 menggunakan operasi push. Elemen-elemen tersebut dimasukkan ke dalam stack sesuai dengan prinsip LIFO.

Selanjutnya, program melakukan operasi pop untuk mengeluarkan elemen teratas dari stack, kemudian menampilkan elemen teratas menggunakan peak. Terakhir, seluruh isi stack ditampilkan menggunakan method print untuk melihat sisa elemen setelah dilakukan operasi penghapusan.

2.4.3 SiswaStack_2511533031

```

1 package pekan3_2511533031;
2 import java.util.*;
3
4 class Siswa_2511533031 {
5     String nama;
6     int nim;
7
8     public Siswa_2511533031(String nama, int nim) {
9         this.nama = nama;
10        this.nim = nim;
11    }
12
13    @Override
14    public String toString() {
15        return "Nim: " + nim + ", Nama: " + nama;
16    }
17 }
18
19 public class SiswaStack_2511533031 {
20     private ArrayList<Siswa_2511533031> stack;
21
22     public SiswaStack_2511533031() {
23         stack = new ArrayList<>();
24     }
25
26     public void push(Siswa_2511533031 mhs) {
27         stack.add(mhs);
28     }
29     public Siswa_2511533031 pop() {
30         if (!isEmpty_3031()) {
31             return stack.remove(stack.size() - 1);
32         }
33         return null;
34     }
35
36     public Siswa_2511533031 peek() {
37         if (!isEmpty_3031()) {
38             return stack.get(stack.size() - 1);
39         }
40         return null;
41     }
42
43     public boolean isEmpty_3031() {
44         return stack.isEmpty();
45     }
46
47     public void tampilkanSiswa_3031() {
48         for (int i = stack.size() - 1; i >= 0; i--) {
49             System.out.println(stack.get(i));
50         }
51     }
52
53     public static void main(String[] args) {
54         SiswaStack_2511533031 studentStack = new SiswaStack_2511533031();
55
56         Siswa_2511533031 mhs1 = new Siswa_2511533031("Ali", 1);
57         Siswa_2511533031 mhs2 = new Siswa_2511533031("Boby", 2);
58         Siswa_2511533031 mhs3 = new Siswa_2511533031("Charles", 3);
59
60         studentStack.push(mhs1);
61         studentStack.push(mhs2);
62         studentStack.push(mhs3);
63
64         System.out.println("Siswa di dalam stack:");
65         studentStack.tampilkanSiswa_3031();
66
67         System.out.println("siswa teratas " + studentStack.peek());
68         System.out.println("mengeluarkan siswa teratas dari stack: " + studentStack.pop());
69         System.out.println("daftar siswa setelah di pop :");
70         studentStack.tampilkanSiswa_3031();
71     }
72 }

```

Gambar 2.3

Program SiswaStack_2511533031 merupakan implementasi struktur data **Stack** menggunakan ArrayList untuk menyimpan objek bertipe Siswa_2511533031. Class Siswa digunakan untuk merepresentasikan data mahasiswa yang terdiri dari atribut nama dan nim, serta method toString untuk menampilkan data dalam bentuk string. Pada class SiswaStack, ArrayList digunakan sebagai media penyimpanan yang bersifat dinamis, sehingga memudahkan dalam pengelolaan data.

Program ini menyediakan operasi dasar stack seperti push untuk menambahkan data, pop untuk menghapus elemen teratas, dan peek

untuk melihat elemen teratas tanpa menghapusnya. Selain itu, terdapat method `tampilkanSiswa` untuk menampilkan seluruh isi stack dari atas ke bawah. Pada method utama, beberapa objek siswa dimasukkan ke dalam stack, kemudian ditampilkan, diambil elemen teratasnya, serta dilakukan penghapusan untuk menunjukkan cara kerja stack dengan prinsip LIFO.

2.4.4 StackPostFix_2511533031

```

1 package pekan3_2511533031;
2 import java.util.*;
3 public class StackPostfix_2511533031 {
4     public static int postfixEvaluate_3031(String expression) {
5         Stack<Integer> s = new Stack<Integer>();
6         Scanner input = new Scanner(expression);
7         while (input.hasNext()) {
8             if (input.hasNextInt()) {
9                 s.push(input.nextInt());
10            } else {
11                String operator = input.next();
12                int operand2 = s.pop();
13                int operand1 = s.pop();
14                if (operator.equals("+")) {
15                    s.push(operand1 + operand2);
16                } else if (operator.equals("-")) {
17                    s.push(operand1 - operand2);
18                } else if (operator.equals("*")) {
19                    s.push(operand1 * operand2);
20                } else {
21                    s.push(operand1 / operand2);
22                }
23            }
24        }
25        input.close();
26        return s.pop();
27    }
28    public static void main(String[] args) {
29        System.out.println("hasil postfix= "+postfixEvaluate_3031("5 2 4 * + 7 -"));
30    }
31 }
32 }
33 }
34 }

```

Gambar 2.4

Program `StackPostfix_2511533031` digunakan untuk menghitung hasil dari ekspresi postfix menggunakan struktur data **Stack**. Metode `postfixEvaluate` menerima input berupa string ekspresi postfix, kemudian memprosesnya menggunakan objek `Stack`. Program membaca setiap elemen dalam ekspresi, jika berupa angka maka langsung dimasukkan ke dalam stack, sedangkan jika berupa operator maka dilakukan pengambilan dua elemen teratas untuk dilakukan operasi perhitungan.

Operasi yang didukung dalam program ini meliputi penjumlahan, pengurangan, perkalian, dan pembagian. Hasil dari setiap operasi akan dimasukkan kembali ke dalam stack hingga seluruh ekspresi selesai diproses. Pada akhirnya, nilai yang tersisa di dalam stack merupakan hasil akhir dari perhitungan postfix.

2.4.5 NilaiMaksimum_2511533031

```

1 package pekan3_2511533031;
2 import java.util.*;
3 public class NilaiMaksimum_2511533031 {
4
5     public static int max(Stack<Integer> s) {
6         Stack<Integer> backup = new Stack<Integer>();
7         int maxValue = s.pop();
8         backup.push(maxValue);
9         while (!s.isEmpty()) {
10             int next = s.pop();
11             backup.push(next);
12             maxValue = Math.max(maxValue, next);
13         }
14         while (!backup.isEmpty()) {
15             s.push(backup.pop());
16         }
17         return maxValue;
18     }
19
20     public static void main(String[] args) {
21         Stack<Integer> s = new Stack<Integer>();
22         s.push(70);
23         s.push(12);
24         s.push(20);
25         System.out.println("isi stack " + s);
26         System.out.println("Stack Teratas " + s.peek());
27         System.out.println("Nilai maksimum " + max(s));
28     }
29 }
30

```

Gambar 2.5

Program NilaiMaksimum_2511533031 digunakan untuk mencari nilai maksimum dari elemen yang terdapat dalam sebuah **Stack**. Method max bekerja dengan mengambil satu per satu elemen dari stack menggunakan operasi pop, kemudian membandingkannya untuk mendapatkan nilai terbesar. Selama proses ini, elemen yang diambil disimpan sementara ke dalam stack lain bernama backup agar data tidak hilang.

Setelah seluruh elemen diperiksa dan nilai maksimum diperoleh, program mengembalikan isi stack ke kondisi semula dengan memindahkan kembali elemen dari backup ke stack utama. Pada method main, beberapa nilai dimasukkan ke dalam stack, kemudian ditampilkan isi stack, elemen teratas, serta nilai maksimum yang diperoleh.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa **Stack** merupakan struktur data yang menggunakan prinsip LIFO (Last In First Out), dimana elemen terakhir yang masuk akan menjadi elemen pertama yang keluar. Stack memiliki operasi dasar seperti push, pop, dan peek yang digunakan untuk mengelola data.

Melalui praktikum ini, mahasiswa telah mampu mengimplementasikan stack menggunakan berbagai pendekatan seperti array dan ArrayList, serta menerapkannya dalam berbagai kasus seperti pengolahan data, pencarian nilai maksimum, dan evaluasi ekspresi postfix. Dengan demikian, pemahaman tentang stack dapat membantu dalam menyelesaikan berbagai permasalahan dalam pemrograman.

DAFTAR PUSTAKA

- [1] TutorialsPoint, "Stack Data Structure," TutorialsPoint, 2024. [Online]. Available:
https://www.tutorialspoint.com/data_structures_algorithms/stack_algorithm.htm. [Accessed 18 April 2026].
- [2] Programiz, "Stack Data Structure," Programiz, 2024. [Online]. Available:
<https://www.programiz.com/dsa/stack>. [Accessed 18 April 2026].

LAPORAN TUGAS

A. Kode Program

Berikut kode program yang telah dibuat pada tugas praktikum pekan-3:

1. Website_2511533031

```

1 package pekan3_2511533031;
2
3 public class Website_2511533031 {
4     private String judul_3031;
5     private String url_3031;
6
7     // Constructor
8     public Website_2511533031(String judul, String url) {
9         this.judul_3031 = judul;
10        this.url_3031 = url;
11    }
12
13    // Getter
14    public String getJudul_3031() {
15        return judul_3031;
16    }
17
18    public String getUrl_3031() {
19        return url_3031;
20    }
21
22    // Setter
23    public void setJudul_3031(String judul) {
24        this.judul_3031 = judul;
25    }
26
27    public void setUrl_3031(String url) {
28        this.url_3031 = url;
29    }
30 }

```

Class Website_2511533031 merupakan Abstract Data Type (ADT) yang digunakan untuk merepresentasikan data sebuah website dalam program simulasi browser history. Class ini berfungsi sebagai blueprint atau cetakan untuk membuat objek website yang berisi informasi penting seperti judul dan URL. Dengan menggunakan class, data yang berkaitan dapat disatukan dalam satu objek sehingga lebih terstruktur, rapi, dan mudah dikelola dalam program.

Class ini juga dilengkapi dengan constructor, getter, dan setter sebagai sarana untuk mengelola data. Constructor digunakan untuk menginisialisasi nilai atribut saat objek dibuat, sehingga setiap objek website langsung memiliki data yang lengkap. Getter digunakan untuk mengambil nilai dari atribut, sedangkan setter digunakan untuk mengubah nilai atribut jika diperlukan.

2. Browser_2511533031

```

1 package pekan3_2511533031;
2
3 import java.util.Scanner;
4 import java.util.Stack;
5
6 public class Browser_2511533031 {
7     public static void main(String[] args) {
8         Stack<Website_2511533031> history = new Stack<>();
9         Scanner input = new Scanner(System.in);
10
11         int pilihan;
12
13         do {
14             System.out.println("=== Browser History NIM: 2511533031 ===");
15             System.out.println("1. Kunjungi Website (Push)");
16             System.out.println("2. Tombol Back (Pop)");
17             System.out.println("3. Lihat Halaman Aktif (Peek)");
18             System.out.println("4. Cek Status History");
19             System.out.println("5. Keluar");
20             System.out.print("Pilihan: ");
21             pilihan = input.nextInt();
22             input.nextLine();
23
24             switch (pilihan) {
25                 case 1:
26                     System.out.print("Masukkan Judul: ");
27                     String judul = input.nextLine();
28
29                     System.out.print("Masukkan URL: ");
30                     String url = input.nextLine();
31
32                     history.push(new Website_2511533031(judul, url));
33                     System.out.println("Berhasil mengunjungi halaman!");
34                     break;
35
36                 case 2:
37                     if (!history.isEmpty()) {
38                         Website_2511533031 removed = history.pop();
39                         System.out.println("Kembali dari: " + removed.getJudul_3031());
40                     } else {
41                         System.out.println("History kosong, tidak bisa kembali.");
42                     }
43                     break;
44
45                 case 3:
46                     if (!history.isEmpty()) {
47                         Website_2511533031 current = history.peek();
48                         System.out.println("Halaman saat ini:");
49                         System.out.println("Judul: " + current.getJudul_3031());
50                         System.out.println("URL: " + current.getUrl_3031());
51                     } else {
52                         System.out.println("Belum ada halaman yang dibuka.");
53                     }
54                     break;
55
56                 case 4:
57                     if (history.isEmpty()) {
58                         System.out.println("History kosong.");
59                     } else {
60                         System.out.println("Jumlah history: " + history.size());
61                     }
62                     break;
63
64                 case 5:
65                     System.out.println("Keluar...");
66                     break;
67
68                 default:
69                     System.out.println("Pilihan tidak valid.");
70             }
71         } while (pilihan != 5);
72     }
73 }
74 }

```

Program ini merupakan implementasi sederhana dari konsep **stack** (**tumpukan**) untuk membuat fitur *browser history*. Di dalamnya digunakan `Stack<Website_2511533031>` bernama `history` untuk menyimpan data halaman web yang dikunjungi. Setiap halaman disimpan sebagai objek dari class `Website_2511533031` yang kemungkinan berisi atribut seperti judul

dan URL. Program juga menggunakan Scanner untuk menerima input dari pengguna melalui menu interaktif di console.

Di awal program, pengguna akan disajikan menu utama dalam perulangan do-while yang terus berjalan sampai pengguna memilih keluar. Variabel pilihan digunakan untuk menampung input menu. Setiap input kemudian diproses menggunakan switch-case untuk menentukan aksi yang dilakukan, seperti mengunjungi website, kembali ke halaman sebelumnya, melihat halaman aktif, mengecek jumlah history, atau keluar dari program.

Pada **case 1**, program melakukan operasi *push*, yaitu menambahkan objek website baru ke dalam stack berdasarkan input judul dan URL dari pengguna. Ini menandakan pengguna “mengunjungi halaman baru”. Data tersebut langsung dimasukkan ke dalam `history.push()`.

Pada **case 2**, dilakukan operasi *pop*, yaitu menghapus elemen paling atas dari stack (halaman terakhir yang dikunjungi). Sebelum dihapus, data halaman tersebut diambil dan ditampilkan sebagai halaman yang sedang ditinggalkan. Jika stack kosong, program memberi pesan bahwa history tidak bisa kembali.

Pada **case 3**, digunakan operasi *peek* untuk melihat elemen paling atas tanpa menghapusnya. Ini merepresentasikan halaman yang sedang aktif saat ini. Program menampilkan judul dan URL dari halaman tersebut. Jika stack kosong, maka ditampilkan pesan bahwa belum ada halaman yang dibuka.

Pada **case 4**, program mengecek kondisi stack menggunakan `isEmpty()` dan `size()`. Jika kosong, ditampilkan pesan bahwa history kosong. Jika tidak, program menampilkan jumlah halaman yang tersimpan dalam history.

Terakhir, **case 5** digunakan untuk keluar dari program dengan menghentikan perulangan. Sedangkan default digunakan untuk menangani input yang tidak sesuai menu.